

DaemonCore Academy Range Registry Guide

Registration, containment verification, operation, and recovery

DOCUMENT	RELEASE	STATUS	PUBLISHED
DC-DOC-900-B	9.0.0	Official	14 September 2026

Intended audience Operators, instructors, lab administrators, and support teams

This guide explains how DaemonCore discovers registered range packs, verifies their integrity, proves containment, launches disposable services, and rejects unsafe or altered environments.

Registry model

The range registry is the content-addressed catalog in `ranges/index.json`. Each entry identifies a shipped pack and its integrity digest. A pack is not trusted merely because its directory exists. DaemonCore compares the registered tree with the packaged files before orchestration.

Web Forge, Enterprise Forge, classic ranges, and Living Range use different orchestration paths but share the same release principle: the desktop owns lifecycle, validates the boundary, and records the exact run contract.

Registry element	Purpose
Pack identifier	Stable reference used by the desktop and mission catalog
Content digest	Detects missing, added, or modified range files
Compose definition	Declares internal services and operator environment
Runtime contract	Binds seed, mode, topology, objectives, and policy
Receipt	Records verified containment and run identity

Containment acceptance

DaemonCore must observe an internal-only Docker network, zero host mounts, zero published ports, no privileged containers, and failed internet-egress probes before presenting a range as sealed. A missing or inconclusive proof is a launch failure, not a warning.

Control	Required value	Failure response
Internet egress	Blocked	Stop launch and destroy the range
Host mounts	0	Reject the pack
Published ports	0	Reject the pack
Privileged containers	0	Reject the pack
Range integrity	Digest match	Refuse orchestration

Launch and teardown

Confirm Docker is running and accessible to the current user. Select the scenario and mode, launch the range, and wait for the containment receipt before entering commands. Use the in-app teardown control when finished. Automatic cleanup also runs during normal application shutdown.

Do not manually attach public networks, publish ports, mount host directories, or start privileged containers to force a failed range to work. Those changes invalidate the range boundary and its results.

Troubleshooting

Symptom	Likely cause	Correct response
Docker offline	Engine stopped or inaccessible	Start Docker and rerun readiness checks
Integrity mismatch	Range files changed	Restore the official package; do not bypass verification
Egress check fails	Network is not internal-only	Destroy the range and inspect Docker networks
Console cannot resolve services	Range startup incomplete	Wait for health checks, then relaunch
Stale containers	Interrupted prior shutdown	Use the app teardown flow and verify cleanup

Registry maintenance

Release maintainers must rebuild ranges/index.json after an intentional range-file change, rerun integrity verification, exercise the applicable Docker matrix, and package the regenerated registry with the same application version. Operators should never regenerate the registry to legitimize an unreviewed local modification.

Document control

This document describes DaemonCore Academy 9.0.0. When application behavior and this guide differ, stop the affected workflow and consult the release notes or support channel before continuing. Product and framework names belonging to third parties are used descriptively and do not imply endorsement.

Field	Value
Owner	DaemonCore Apps
Release	9.0.0
Classification	Public product documentation
Review trigger	Material product, policy, or credential-status change